

INFR11287 Advanced Topics in Natural Language Processing

Mock Examination Paper 3: Mark Scheme

This mark scheme gives indicative points. Award credit for equivalent answers that demonstrate the same understanding.

1. Vision-language assistant. Total: 15 marks.

- (a) The task requires connecting language to visual objects, regions, text in images, and situated context. It addresses grounding because meanings of words such as “this”, labels, artefacts, and spatial relations depend on perception rather than text alone. Award up to 3 marks.
- (b) Components: vision encoder converts image/patches into visual features; projector/connector maps visual features into the LLM hidden/token space; backbone LLM conditions on visual tokens and text to generate answers. Award up to 4 marks.
- (c) Linear connector: cheap and stable but less expressive. MLP: more expressive with moderate cost and possible overfitting/initialisation issues. Attention-based connector: can select salient visual information but is costlier and harder to train. Award up to 3 marks.
- (d) Data: exhibit images with captions, OCR labels, VQA pairs, referring expressions, multilingual labels, accessibility-oriented egocentric photos, expert-curated museum facts. Risks: licensing, privacy of visitors, cultural misrepresentation, hallucinated labels, domain shift from generic datasets. Award up to 3 marks.
- (e) Intrinsic: captioning, OCR, VQA, object recognition, hallucination benchmarks. Extrinsic: visitor task success, answer usefulness, accessibility studies with visually impaired users, latency. Ethics/accessibility: wrong descriptions, cultural harms, privacy, overconfident unsafe guidance. Award up to 2 marks.

2. Agents and computer use. Total: 15 marks.

- (a) An agent perceives an environment through sensors and acts through actuators. API design: observations are tool outputs/form states and actions are API calls. Browser design: observations are screenshots/OCR/accessibility trees and actions are clicks, typing, scrolling, navigation, and stopping. Award up to 3 marks.
- (b) Text-only/API agent: more reliable and structured when APIs exist, easier validation; disadvantage is limited to available tools and schemas. Vision-based CUA: can operate across arbitrary GUIs; disadvantage is visual grounding, UI fragility, and action risk. Award up to 4 marks.
- (c) I is the instruction/goal; s_t is the environment state at step t , often a screenshot; a_t is the action such as click/type/scroll/terminate. History matters for tracking progress, avoiding repeated actions, recovering from mistakes, and interpreting state changes. Award up to 3 marks.
- (d) Failure modes: clicking wrong button, stale UI after page changes, entering private data in wrong field, loops, irreversible cancellation/payment, prompt injection in webpage text. Mitigations: constrained action space, confirmations, sandboxing, API preference, state checks, logging, time/action limits, human approval for irreversible actions. Award up to 3 marks.
- (e) Final success alone hides path quality and risk. Additional measures: intermediate progress, visual grounding accuracy, loop rate, number of actions, recovery/backtracking, unsafe action rate, latency, robustness to UI changes. Award up to 2 marks.

3. MCQ explanations. Total: 20 marks.

- (a) **B.** A vision encoder outputs visual feature vectors, while the LLM expects vectors in its own hidden/token embedding space. The connector or projector maps between these spaces so image information can be used as visual tokens or prefix representations. It is a cross-modal interface, not a storage or tokenisation replacement.

- (b) **C.** In a Vision Transformer, images are split into patches, and each patch becomes a visual token. Higher resolution or smaller patch size usually means more patches, so the sequence is longer and attention/computation can increase. More visual tokens may help detail but cost more.
- (c) **A.** A computer-use agent needs a termination action so it can indicate that the task is complete and stop acting. Without this, the agent may loop, continue unnecessary actions, or damage the state after success. Termination does not make actions reversible.
- (d) **A.** Watermarking biases token choice toward detectable green-list tokens, but low-entropy states may have only one or very few plausible next tokens. Forcing a watermark there can hurt fluency or factuality, while not forcing it weakens detectability. This is a core quality-detectability tradeoff.
- (e) **A.** Sparse autoencoders take dense model activations and learn a sparse, often overcomplete, feature representation. This can reveal more candidate features than the original activation dimension because features may be represented in superposition. SAEs do not prove every feature is cleanly monosemantic.
- (f) **A.** Superposition is the idea that neural models can represent more features than they have dimensions by overlapping feature directions. This helps explain why individual neurons can be polysemantic. It is a representation phenomenon, not a parsing output.
- (g) **B.** Dependency parsing represents syntactic structure as head-dependent relations between words. This contrasts with constituency parsing, which groups words into nested phrases. Dependency structures are often evaluated with attachment scores.
- (h) **A.** When a model generates a linearised constituency tree, it may produce unmatched brackets or brackets whose nonterminal labels do not correspond. This is a structural validity issue caused by treating tree generation as sequence generation. Post-processing can help but may not fix typed-bracket mismatches.
- (i) **A.** Allocational harm concerns unequal access to opportunities, resources, or services, such as worse model quality or decisions for a protected group. Representational harm concerns stereotypes, denigration, erasure, or biased portrayals. The distinction matters for choosing evaluation and mitigation strategies.
- (j) **C.** A finite-state machine has finite memory and can recognise regular languages, but it lacks unbounded stack-like memory for nested or recursively structured dependencies. Pushdown automata and richer formalisms can model such structure more naturally. This limitation is why FSMs are weaker than models with unbounded memory.